

SINTAXE DE RECURSOS DO TEMPLATE MARICARAI

1. Códigos de Controles

1. ~ switch tString-literals on/off
2. ^A Zera todos os campos
3. ^B Indica que os caracteres seguintes contém o nome do campo
4. ^C O campo corrente possui uma lista de opções do mesmo tipo.
5. ^D Use o próximo caractere como um delimiter de campo
6. ^E Campo do tipo enumerado.
7. ^F Usado para criar restrições e relacionamentos
8. ^G Usada para concatenar duas listas do tipo PSItem.
9. ^H Campo escondido
10. ^I Link para cadeia de template pSItem
11. ^J Retorno do carro
12. ^k Os caracteres após ^k é capturado no campo TDmxFieldRec.Default
13. \k k minuscuro tipo FldDbRadioButton.
14. \K K maiúsculo tipo FldRadioButton.
15. ^L Link para uma URL ou actionItens
16. ^M Fim da linha
17. ^N A sequência a seguir é o hint do campo.
18. ^O Campo fldBLOb
19. ^P Usado para controlar o flag do tipo de campo
20. ^R Campo somente de leitura
21. ^S Salte campo para o próximo campo de acesso normal
22. ^T O campo é um botão de ação
23. ^U Informar um limite superior campos do tipo byte. Faixa: [0..255]
24. ^V Se o campo for numérico, preencha com '#0'(AccNormal) se for alfanumérico, preencha com ' ' AccNormal
25. ^X Campo de BOOLEAN especial
26. ^Z Zera se este campo está vazio

2. Delimitadores de campos:

1. #0 = delimiter de campo técnico (não exibe)
2. \ = exibe como um espaço
3. | = exibe como

3. Tipos de campos:

1. Campo string com tamanho da string no byte 0:

1. Const fldStr = 'S';

1. O caractere S informa ao componente **TUiDmxScroller** que a sequência de caracteres S após o caractere "\" representa no

buffer do formulário um tipo *ShortString* que só aceita caracteres maiúsculas.

1. EXEMPLO

[illegible]

2. **Const fldstr_Lowcase = 's';**

1. O caractere `s` informa ao componente **TUiDmxScroller** que a sequência de caracteres `s` após o caractere `"\"` representa no buffer do formulário um tipo `ShortString` que só aceita caractere minúscula.

1. EXEMPLE

[illegible]

3. Const fldSTRNUM = '#'

1. O caractere # usado na máscara do template e informa ao componente **TUiDmxScroller** que a sequência de caracteres # após o caractere "\" representa no buffer do formulário um tipo ShortString que só aceita caracteres numéricos.

1. EXEMPLE

Resourcestring

```
~Telefone: ~\((##) # #####-##### //85 9 9702  
4498  
~Cep:      ~\### ##### ###
```

2. Campo string com tamanho da string limitado pelo caractere 0 (zero) no fim da string.

1. **Const fldAnsiChar = 'C'**

1. O caractere **c** usado na máscara do Template, informa ao componente **TUiDmxScroller** que a sequência de caracteres 'C' após o caractere "\" representa no buffer do formulário um tipo AnsiString que só aceita caracteres maiúsculos.

1. EXEMPLO

```
Resourcestring

~Nome:      ~'\CCCCCCCCCCCCCCCCCCCC' //Campo com
caracteres maiúsculas

~Endereço:~'\CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
//Campo com 40 caractere maiúsculas
```

2. **Const fldAnsiChar_LowCase = 'c'**

1. O caractere **c** usado na máscara do Template, informa ao componente **TUiDmxScroller** que a sequência de caracteres **c** após o caractere "\" representa no buffer do formulário um tipo AnsiString que só aceita caractere minúsculo.

1. EXEMPLO

```
Resourcestring

~Nome:      ~'\cccccccccccccccccccc' //Campo com
caracteres minúsculas

~Endereço:~'\cccccccccccccccccccccccccccccccccccc
//Campo com 40 caractere minúsculas
```

3. **Const fldAnsiCharNUM = '0'**

1. Usado na máscara do Template, informa ao componente **TUiDmxScroller** que a sequência de caracteres 0 após o caractere "\" representa no buffer do formulário um tipo AnsiString que só aceita caractere numérico ['0'..'9'].

1. EXEMPLO

```
Resourcestring

~telefone :~\((00) 0 0000-0000' //85 9 9702
4498
```

4. **Const fldAnsiCharVAL = 'N';**

1. O caractere *N* usado na máscara do template informa ao componente **TUiDmxScroller** que a sequência de caracteres *0* após o caractere "\" representa no buffer do formulário um tipo AnsiString que só aceita caractere numérico [*0'..'9'*] com formatação dbase.

1. **EXEMPLO**

```
Resourcestring  
  
'~telefone :~\ (NN) N NNNN-NNNN' //85 9  
9702 4498
```

3. Campos do tipo inteiro

1. **Const fldBYTE = 'B';**

1. O caractere *B* usado na máscara do template, informa ao componente **TUiDmxScroller** que a sequência de caracteres *B* após o caractere "\" representa no buffer do formulário um tipo byte e só pode receber valor na faixa [*0..255*].

1. **EXEMPLO**

```
Resourcestring  
  
'~idade :~\ BBB' //Os três dígitos estarão em um  
buffer de 1 byte;
```

2. **Const fldSHORTINT = 'J';**

1. O caractere *J* usado na máscara do template, informa ao componente **TUiDmxScroller** que a sequência de caracteres *J* após o caractere "\" representa no buffer do formulário um tipo byte e só pode receber valor na faixa [*-128 a 127*], usado quando precisamos de um tipo byte com valores negativos.

1. **EXEMPLO**

```
Resourcestring  
  
'~Tempo de vida ou de morte:~\ JJJ' //Os  
três dígitos estarão em um buffer de 1 byte;
```

3. **Const fldSmallWORD = 'W'**

1. O caractere *W* usado na máscara do template, informa ao componente **TUiDmxScroller** que a sequência de caracteres *W* após o caractere "\" representa no buffer do formulário um tipo word curto e só pode receber valor na faixa [0 a 65535].

1. EXEMPLO

```
Resourcestring  
  
'~idade :~\WW,WWW' //0s cinco dígitos  
estarão em um buffer de 2 bytes;
```

4. **Const fldSmallInt = 'I';**

1. O caractere *I* usado na máscara do template, informa ao componente **TUiDmxScroller** que a sequência de caracteres *I* após o caractere "\" representa no buffer do formulário um tipo Inteiro curto (2 bytes) e só pode receber valor na faixa [-32.768 a 32.767].

1. EXEMPLO

```
Resourcestring  
  
'~Tipo small int :~\II,III' //0s cinco  
dígitos estarão em um buffer de 2 bytes;
```

5. **Const fldLONGINT = 'L';**

1. O caractere *L* usado na máscara do template, informa ao componente **TUiDmxScroller** que a sequência de caracteres *L* após o caractere "\" representa no buffer do formulário um tipo Inteiro longo (4 bytes) e só pode receber valor na faixa [-2.147.483.648 a 2.147.483.647].

1. EXEMPLO

```
Resourcestring  
  
'~Tipo longint :~\LLL,LLL' //0s seis  
dígitos estarão em um buffer de 4 bytes;
```

6. **Const fldBoolean = 'X';**

1. O caractere *X* usado na máscara do template, informa ao componente **TUiDmxScroller** que o campo é do tipo byte e só pode ter dois valores.

1. NOTAS

1. Valores possíveis:

1. 0 = False

2. 1 = True

3. A forma de editá-los deve ser com o componente checkbox.

2. EXEMPLO

Resourcestring

```
'~Aceita os termos do contrato~\X
^Bfld_ceita_contrato^Aceita os termos do
contrato?';
```

7. Const FldRadioButton = 'K'; //Maiúscula

1. O caractere *K* (maiúsculo) usado na máscara do Template, , informa ao componente **TUiDmxScroller** que é um campo tipo TCluster e é representado no template por um controle TRadioButton.

1. Um Template pode conter vários campos do tipo cluster e o mesmo é identificado após a sequencia **\Kx**, onde *x* indica que a informação pertence ao campo *x*,

1. EXEMPLO

Resourcestring

```
~ SEXO ~
~ ~\Ka Masculino
~ ~\Ka Feminino
~ ~\Ka Indefinido
~ ESTADO CIVIL
~ ~\Kb Solteiro
~ ~\Kb Casado
~ ~\Kb Divorciado
```

2. NOTA

1. Os campos *cluster* possuem o mesmo número do campo e na primeira ocorrência contém o nome do campo na lista *pDmxFieldRec*.

4. Campos do tipo Real

1. fldExtended = 'E';

1. O caractere *E* usado na máscara do template, informa ao componente **TUiDmxScroller** que a sequência de caracteres *E* após o caractere "\" representa no buffer do formulário um tipo Extended (10 bytes) e pode receber valores positivos e negativos.

1. EXEMPLO

```
Resourcestring  
  
    '~Patrimônio líquido do grupo  
    :~\EEE,EEE,EEE,EEE.EE' //Os 14 dígitos estarão  
    em um buffer de 10 bytes;
```

2. Const fldRealNum = 'R';

1. O caractere *R* usado na máscara do template, informa ao componente **TUiDmxScroller** que a sequência de caracteres *R* após o caractere "\" representa no buffer do formulário um tipo double (8 bytes) e pode receber valores positivos e negativos.

1. EXEMPLO

```
Resourcestring  
  
    '~          Patrimônio líquido  
    :~\RRR,RRR,RRR.RR' //Os 11 dígitos estarão em um  
    buffer de 8 bytes;
```

3. Const fldRealNum_Positivo = 'r';

1. O caractere *r* usado na máscara do template, informa ao componente **TUiDmxScroller** que a sequência de caracteres *r* após o caractere "\" representa no buffer do formulário um tipo double (8 bytes) e só pode receber valores positivos.

1. EXEMPLO

```
Resourcestring  
  
    '~Patrimônio líquido :~\RRR,RRR,RRR.RR'  
    //Os 11 dígitos estarão em um buffer de 8 bytes;
```

4. Const fldReal4 = 'O';

1. O caractere *O* usado na máscara do template, informa ao componente **TUiDmxScroller** que a sequência de caracteres *O* após o caractere "\" representa no buffer do formulário um tipo single (4 bytes) e só pode receber valores positivos.

1. **EXEMPLO**

```
Resourcestring  
  
'~Salário :~\00,000.00' //0s 7 dígitos  
estarão em um buffer de 4 bytes;
```

5. **fldReal4Positivo = 'o';**

1. O caractere *o* usado na máscara do template, informa ao componente **TUiDmxScroller** que a sequência de caracteres *o* após o caractere "\" representa no buffer do formulário um tipo single (4 bytes) e pode receber valores positivos e negativos.

1. **EXEMPLO**

```
Resourcestring  
  
'~Patrimônio líquido :~\00,000.00' //0s 7  
dígitos estarão em um buffer de 4 bytes;
```

6. **fldReal4P = 'P';**

1. O caractere *P* usado na máscara do template, informa ao componente **TUiDmxScroller** que a sequência de caracteres *P* após o caractere "\" representa no buffer do formulário um tipo single (4 bytes) com a seguinte característica: Ao ler compo do arquivo o resultado é multiplicado por 100 e ao gravar é dividido 100 usado para campos relativos e pode conter valores positivos e negativos.

1. **EXEMPLO**

```
Resourcestring  
  
'~Valor percentual :~\PPP.PP' //0s 5  
dígitos estarão em um buffer de 4 bytes;
```

7. **fldReal4PPositivo = 'p';**

1. O caractere *p* usado na máscara do template, informa ao componente **TUiDmxScroller** que a sequência de caracteres *p* após

o caractere "\" representa no buffer do formulário um tipo single (4 bytes) com a seguinte característica: Ao ler compo do arquivo o resultado é multiplicado por 100 e ao gravar é dividido 100 usado para campos relativos e só pode conter valores positivos.

1. EXEMPLO

Resourcestring

'~Valor percentual :~\ppp.pp' //Os 5
dígitos estarão em um buffer de 4 bytes;